

ASSIGNMENT PART 2

Voronoi Diagrams, Power Diagrams, Optimal Transport,
and Incompressible Euler Equations

Georgy Kuznetsov
École Polytechnique

June 2026

1 Introduction

This document reports the implementation of assignment part 2, concretely particle-based fluid solver, following scheme of Gallouet and Merigot. Solver has three layers, corresponding to each lab:

1. **Lab 6** — Voronoï diagram via Parallel Linear Enumeration and Sutherland–Hodgman polygon clipping.
2. **Lab 7** — Extension to a Power diagram with weights optimised by L-BFGS to solve a semi-discrete optimal transport problem
3. **Lab 8** — Full Euler simulation: partial optimal transport with an air variable, disk-clipping of fluid cells and a spring force driving particles toward their centroids

All code is written in C++ and is available at https://github.com/rumata-feathers/computer_graphics.

2 Voronoi Diagram (Lab 6)

2.1 Definition

Given N sites $\{P_i\}_{i=1}^N \subset \Omega = [0, 1]^2$, the Voronoi cell of site P_i is

$$V_i = \{x \in \Omega : \|x - P_i\|^2 \leq \|x - P_j\|^2, \quad \forall j \neq i\}$$

Every cell is a convex polygon and the cells tile Ω without overlap

2.2 Parallel Linear Enumeration

Each cell V_i is computed independently by starting with the full unit square and successively clipping it against the perpendicular bisector of the segment $[P_i, P_j]$ for every other site P_j . This is easily parallelisable across sites (implemented with `#pragma omp parallel for`). To avoid redundant clipping by distant sites only the $k = 30$ nearest neighbours of each P_i are used; far-away sites cannot affect V_i once nearby bisectors have already been applied

2.3 Sutherland–Hodgman Clipping

The main thread function is `clip_by_bisector`, which clips a convex polygon V by the bisector half-plane $\{x : \|x - P_i\|^2 \leq \|x - P_j\|^2\}$. The algorithm iterates over each directed edge (C, N) of V and for each pair of consecutive vertices considers four cases:

- Both inside $\Rightarrow$ keep N
- C inside N outside $\Rightarrow$ then add edge bisector intersection
- if C outside N inside $\Rightarrow$ add intersection then N
- Both outside $\Rightarrow$ skip

The intersection parameter is

$$t = \frac{\langle M - C, P_i - P_j \rangle}{\langle N - C, P_i - P_j \rangle},$$

where $M = \frac{P_i + P_j}{2}$ is the midpoint The intersection point is $C + t(N - C)$

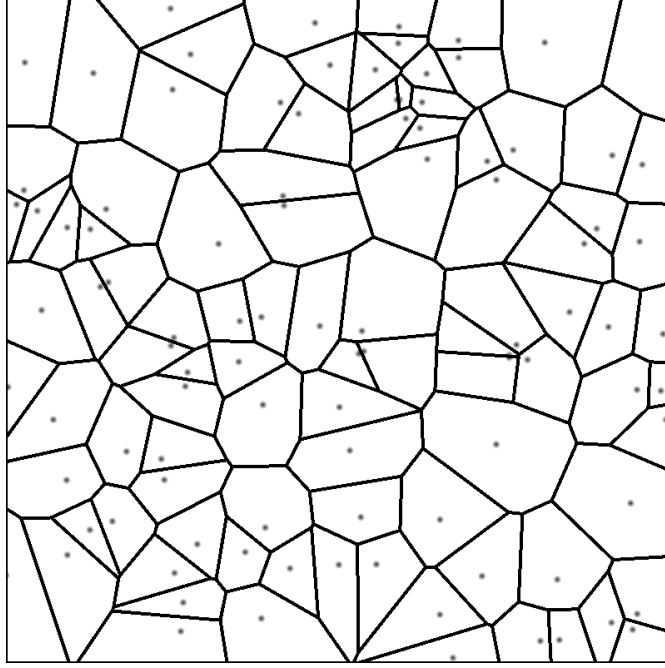


Figure 1: Vorono diagram for $N = 100$ random sites in $[0, 1]^2$

3 Power Diagram (Lab 7)

3.1 Weighted Bisector

Associating a real weight w_i with each site generalises the Voronoidiagram to a Power diagram. The P_i is

$$L_i(\mathbf{w}) = \{ x \in \Omega : \|x - P_i\|^2 - w_i \leq \|x - P_j\|^2 - w_j, \quad \forall j \neq i \}$$

The boundary between L_i and L_j is line, but the midpoint shifts by $\frac{w_i - w_j}{2\|P_i - P_j\|^2}$ along $P_i - P_j$:

$$M_{ij}(\mathbf{w}) = \frac{P_i + P_j}{2} - \frac{w_i - w_j}{2\|P_i - P_j\|^2}(P_i - P_j)$$

The clipping condition becomes a distance test $\|x - P_i\|^2 - w_i \leq \|x - P_j\|^2 - w_j$, implemented directly in `clip_by_bisector` by replacing the standard midpoint with $M_{ij}(\mathbf{w})$. Cells can now be smaller or larger depending on the sign of $w_i - w_j$

4 Optimal Transport via L-BFGS (Lab 7)

4.1 Optimal Transport

We wish to find weights $\mathbf{w}$ such that each power diagram has a prescribed area $\lambda_i = 1/N$ (equal-area partition of Ω). This is an attempt to minimise the functional

$$\mathcal{K}(\mathbf{w}) = - \sum_{i=1}^N \left[\int_{L_i(\mathbf{w})} \|x - P_i\|^2 dx - w_i |L_i(\mathbf{w})| + \lambda_i w_i \right]$$

This functional is a concave function in $\mathbf{w}$ (we minimise its negation) and its gradient is

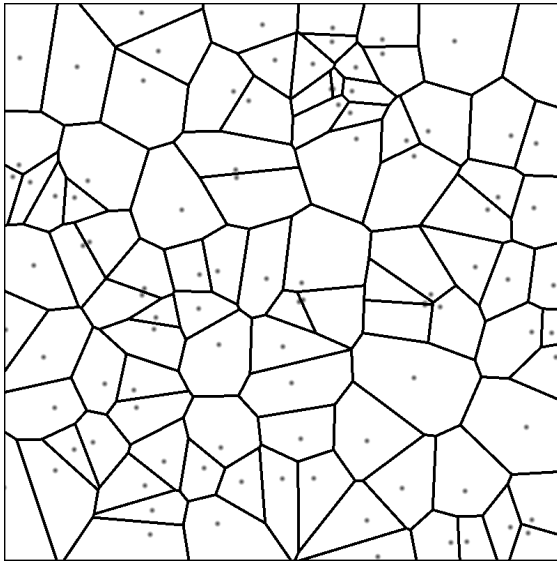
$$\frac{\partial \mathcal{K}}{\partial w_i} = -(\lambda_i - |L_i(\mathbf{w})|)$$

The gradient vanishes when $|L_i(\mathbf{w})| = \lambda_i$ for all i , i.e. at the optimal transport solution

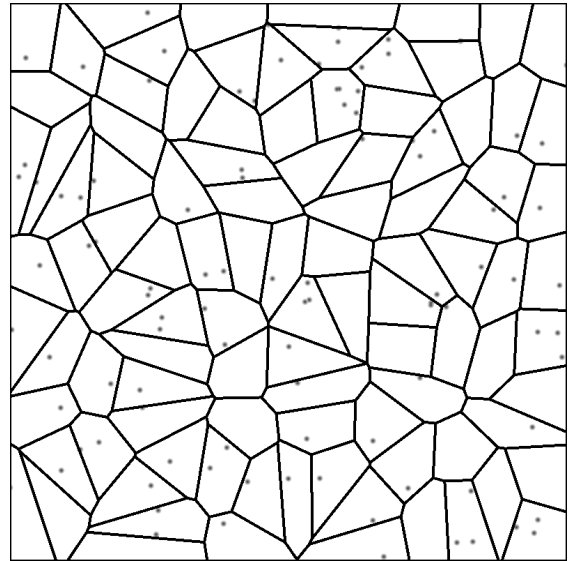
4.2 L-BFGS Solver

Since we are cool to find optimal solution we use the `liblbfgs` library. At each iteration:

1. The current weights are written into `VoronoiDiagram::weights`.
2. `VoronoiDiagram::compute()` recomputes all cells
3. Each cell area and $\int \|x - P_i\|^2 dx$ are computed using the shoelace formula
4. The functional value and gradient vector are returned to L-BFGS



(a) Voronoi cells (unequal areas)



(b) cells after OT (equal areas)

Figure 2: After convergence every cell covers area $1/N$

5 Fluid Simulation (Lab 8)

5.1 Gallouët–Mérigot Scheme

The scheme of de Gallouët and Mérigot discretises the Navier Stokes equation, specifically Euler equation

$$\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \mathbf{g}, \quad \nabla \cdot \mathbf{u} = 0,$$

with a particle method that check compressure w optimal transport

Each fluid particle P_i carries an equal volume fraction $\lambda_i = V_{\text{fluid}}/N$ and incompressibility is requiring $|L_i(\mathbf{w})| = \lambda_i$ at every time step

5.2 Optimal Transport

The fluid does not fill the full domain Ω , because the remaining fraction is “air”. We use OT: the weight vector is extended to $\mathbf{w} \in \mathbb{R}^{N+1}$, where $w_{\text{air}} = w_N$ controls how much space is for air. Functional becomes

$$\mathcal{K}(\mathbf{w}, w_{\text{air}}) = - \sum_{i=1}^N \left[\int_{L_i} \|x - P_i\|^2 dx - w_i |L_i| + \lambda_i w_i \right] - w_{\text{air}} (\lambda_{\text{air}} - A_{\text{air}}),$$

with $\lambda_{\text{air}} = 1 - V_{\text{fluid}}$ and $A_{\text{air}} = 1 - \sum_i |L_i|$ The corresponding gradient components are

$$\frac{\partial \mathcal{K}}{\partial w_i} = -(\lambda_i - |L_i|), \quad \frac{\partial \mathcal{K}}{\partial w_{\text{air}}} = -(\lambda_{\text{air}} - A_{\text{air}}).$$

5.3 Disk Intersection

Within this context it can be seen, that to handle infinite amount of particles one simply need to take intersection of each fluid Laguerre to a disk centered at P_i with radius $r_i = \sqrt{w_i - w_{\text{air}}}$. The disk is approximated by a 64-sided regular polygon, and `clip_by_edge` is applied to each of its edges.

5.4 Time Integration

At each time step $\Delta t = 0.002$ s:

1. Run L-BFGS to find $\mathbf{w}^*$ such that $|\tilde{L}_i(\mathbf{w}^*)| = \lambda_i$ for all i
2. For each particle, compute the centroid C_i of its clipped Laguerre cell $\tilde{L}_i$
3. Apply gravity and a spring force toward the centroid:

$$\mathbf{F}_i = m \mathbf{g} + \frac{1}{\varepsilon^2} (C_i - P_i), \quad \varepsilon = 0.004, \quad m = 200$$

4. Update velocity and position (forward Euler):

$$\mathbf{v}_i \leftarrow \mathbf{v}_i + \frac{\mathbf{F}_i}{m} \Delta t, \quad P_i \leftarrow P_i + \mathbf{v}_i \Delta t$$

5. Elastic bounces when collisions with the boundaries of the unit square

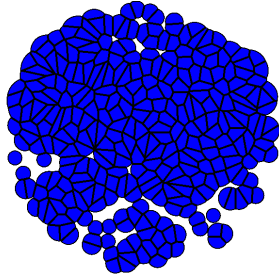
The spring term $\varepsilon^{-2}(C_i - P_i)$ drives each particle toward the centroid of its Laguerre cell producing a pressure-like effect

5.5 Setup and Results

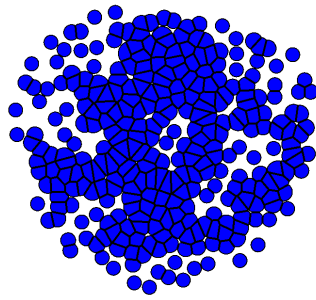
The simulation uses $N = 300$ particles. Initial positions are drawn uniformly in $[0, 1] \times [0, 0.5]$ (lower half of Ω), with zero initial velocities. The fluid volume fraction is $V_{\text{fluid}} = 0.4$. The simulation runs for 1000 frames and is assembled into a video using `ffmpeg`:

```
ffmpeg -framerate 15 -start_number 0 -i test%d.png \
-c:v libx264 -pix_fmt yuv420p output.mp4
```

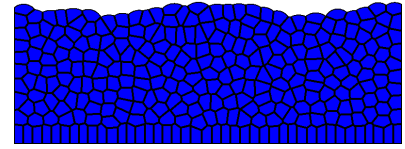
Some parts of the report and ode could have been inspired by work of Julia Korytkowska, Alisa Karlowa and Claude



(a) Frame 0 — initial state



(b) Frame 100 — fluid falls



(c) Frame 500 — steady flow

Figure 3

6 Course Feedback

References

- [1] [lecture notes](#)